University of Bremen
Department of Computer Science
Prof. Dr. h.c. Michael Beetz, PhD

AICOR Institute for Artificial Intelligence
WiSe 2026/2027
Solution for Worksheet 3

Solution: Exercises for the Lecture Advanced Methods of AI

Querying and Reasoning

Tutorial: Thursday, 12.11.2026, 8:30–10:00, room 0.30, ECO 5

Task 1 on paper: 10 minutes in pairs, 5 minutes of discussion. Tasks 2 to 4: about 15 minutes in pairs each, then 5 to 10 minutes of discussion. They need a laptop with a web browser: open the notebook `03-pizza_queries.ipynb` on Binder (link on Stud.IP) and note your predictions and answers on this sheet.

One expression, three readings. Last week you wrote the concept ∃hasTopping.CheeseTopping. In EQL, the query language of KRROOD, the same thing is written

```
D = a(Pizza)(toppings=a(CheeseTopping)())
```

It is a **description**: it denotes a set of objects, as a concept does. It is a **question**: which objects of the knowledge base fit it? It is a **constraint**: a condition that an object has to satisfy, or must not satisfy. Today you use all three readings.

Task 1: What Is a Query?

The interpretation of Worksheet 2. Four pizzas and their toppings; these are *all* the objects and *all* the hasTopping pairs. CheeseTopping, VegetableTopping and MeatTopping are disjoint.

Pizza	Toppings
p1	m1 : CheeseTopping, t1 : VegetableTopping
p2	h2 : MeatTopping, m2 : CheeseTopping
p3	none
p4	v4 : VegetableTopping

(a) One description, several languages. Write D as

i. a description logic concept,
ii. a first-order formula with one free variable x,
iii. an SQL query over the tables `pizza(id)` and `topping(id, pizza_id, kind)`, where `kind` is the name of the topping's class.

Which pizzas of the interpretation are the answer?

Solution

i. `Pizza ⊓ ∃hasTopping.CheeseTopping`, which is C1 of Worksheet 2 restricted to pizzas.
ii. `Pizza(x) ∧ ∃y (hasTopping(x, y) ∧ CheeseTopping(y))`
The free variable is what is asked for. A formula without free variables is a yes/no question.

iii. `SELECT DISTINCT pizza.id FROM pizza JOIN topping ON topping.pizza_id = pizza.id WHERE topping.kind = 'CheeseTopping'`
The pair hasTopping(x, y) has become the foreign key `pizza_id`, and following it has to be written as a join. Without DISTINCT, a pizza with two cheese toppings is returned twice. `WHERE EXISTS (SELECT ...)` is the direct translation of ∃.

Answer: {p1, p2}, the row C1 of last week's table.

For the discussion: two more languages say the same. SPARQL, over the RDF graph of pizza.owl: `SELECT ?x WHERE { ?x a :Pizza ; :hasTopping ?y . ?y a :CheeseTopping }`. Prolog or Datalog: `?- pizza(X), has_topping(X, Y), cheese_topping(Y)`. All of them are syntax for the formula in ii. They differ in what they are asked *of* (Task 4).

(b) Reading EQL. `pizzas` is the list of p1 to p4. What does each query return?

```
p = variable(Pizza, domain=pizzas)      # p ranges over the pizzas
t = flat_variable(p.toppings)           # t ranges over the toppings of p

Q1 = a(Pizza)(toppings=a(MeatTopping)()).from_(pizzas)
Q2 = entity(p).where(for_all(t, HasType(t, VegetableTopping)))
Q3 = set_of(p, t).where(HasType(t, CheeseTopping))
Q4 = the(Pizza)(toppings=a(VegetableTopping)()).from_(pizzas)
```

Solution

- **Q1:** {p2}. A nested description on a list-valued attribute means: *some* element fits.
- **Q2:** {p3, p4}. This is C2 = ∀hasTopping.VegetableTopping. p3 is an answer because it has no toppings: there is nothing to check (vacuous truth).
- **Q3:** the two pairs (p1, m1) and (p2, m2). An answer is a *binding*: one value for each selected variable such that the condition is true. Q1 and Q2 select one variable, so their answers look like objects.
- **Q4:** no answer but an error (MultipleSolutionFound). `the` claims that exactly one object fits, and both p1 and p4 have a vegetable topping. With `a` the answer would be {p1, p4}.

(c) Question, description or constraint? Four people in a pizzeria use D. For each: where do the candidate objects come from, and what counts as the answer?

i. The waiter, to the kitchen display: “Which pizzas on the counter have cheese?”
ii. The menu: “Cheesy pizza: a pizza with a cheese topping.”
iii. A guest: “A pizza with cheese, please.”
iv. The allergy sheet on the wall: “No order of a guest who avoids milk contains a pizza with cheese.”

Solution

i. **Question.** Candidates: the pizzas on the counter, i.e. objects of the knowledge base. Answer: all of them that fit.
ii. **Description.** No candidates at all. The menu defines a concept (a set that depends on the situation); it is terminology, the TBox. It becomes a question as soon as somebody supplies candidates.
iii. **Constraint on an object that does not exist yet.** Candidates: everything the kitchen can make. Answer: *one* new pizza that fits, any one. Everything the guest did not mention is left open; the description is underspecified and the cook fills it in.
iv. **Constraint on the knowledge base.** Candidates: tonight's orders. The answer that everybody hopes for is the empty set; every binding that comes back is a violation, with the guilty order in it.

The expression is the same four times. What differs is where the candidates come from (the *domain*) and what is done with the objects that fit: list them, name them, make one, or raise the alarm.

(d) True or entailed? In Worksheet 2 you answered "does the KB entail φ?" with an argument about *every* model (Task 1b), and "which pizzas belong to C?" by looking at *one* interpretation (Task 2a). Which of the two does a query on p1 to p4 do? What does that mean for the question whether a pizza of which we only know a mozzarella topping is vegetarian?

Solution

The second. The objects *are* the interpretation, and the query is evaluated in it (model checking). This is correct only if the interpretation is complete: whatever is not there is false. That is the closed-world assumption, shared by SQL, Prolog and EQL.

An OWL reasoner does the first: an answer has to hold in every model of the axioms. For the pizza with a mozzarella topping, the query says "vegetarian" (no meat in the list), the reasoner says "not entailed" (there are models in which it also has ham). Evaluation is cheap, polynomial in the data for a fixed query; entailment is expensive. This is one reason why most data lives in databases and not in ontologies.

Task 2: Asking Questions

Notebook, part 2. Thursday evening in a pizzeria: seven pizzas on the counter, three orders, three guests. Run the cells from the top. For every **Predict**, fill in the sheet *before* you run the cell.

(a) One question, two spellings. Write `hot_pizzas` and `hot_orders`. american_hot_1 has two hot toppings. Why is it returned only once?

Solution

```
hot_pizzas = a(Pizza)(toppings=a(PizzaTopping)(spiciness=Spiciness.HOT)).from_(pizzas)
hot_orders = an(Order)(
    pizzas=a(Pizza)(toppings=a(PizzaTopping)(spiciness=Spiciness.HOT))).from_(orders)
```

Results: american_hot_1, and the order of table 1 (Ada). The description says "*some* topping is hot"; an object fits it or does not, however many witnesses there are. The order query passes through two lists: some pizza of the order has some hot topping. In DL: ∃hasPizza.∃hasTopping.∃hasSpiciness.Hot.

(b) *a* or *the*? Predict the result of "the pizza with fish" and of "the pizza with cheese". A robot is told: "Bring me the pizza with cheese." What should it do?

Solution

The first is napoletana_1. The second raises MultipleSolutionFound, because five pizzas have cheese (and NoSolutionFound if none had). "The pizza with cheese" is a definite description: it claims that exactly one such object exists. When the claim is false, the sentence does not refer, and the honest reaction is not to pick one but to ask back, "which one: the Margherita or the Napoletana?". Finding what distinguishes the candidates is again a query.

(c) Some, only, none. Predict with a cross which pizzas each query returns. Then write `vegetarian` (no meat and no fish topping).

Solution

	margherita	napoletana	american_hot	mushroom	veggie	plain	unclosed
Q1 only vegetable toppings					×	×	
Q2 [a mozzarella, a tomato]	×						
Q3 no cheese topping					×	×	

- **Q1:** plain_1 has no toppings, so "all of them are vegetables" is vacuously true. It is p3 of Worksheet 2.
- **Q2:** a *list* of descriptions describes the whole list: first a mozzarella, then a tomato, nothing else. mushroom_1 has both and a third topping, unclosed_1 has only the mozzarella. A single description means "some element"; the list is "some + only", the closure axiom of Worksheet 2, in one expression.
- **Q3:** nobody stated "veggie_1 has no cheese". The query concludes it because none is listed: negation as failure.

```
vegetarian = entity(p).where(not_(exists(t, or_(HasType(t, MeatTopping), HasType(t, FishTopping)))))
```

Result: margherita_1, mushroom_1, veggie_1, plain_1, unclosed_1. Equivalent: `for_all(t, not_(or_(...)))`. Use one `exists`: if two quantifiers share the variable `t`, it is no longer local to either of them and the query becomes a join. Two quantifiers need two `flat_variable(p.toppings)`.

(d) Answers are bindings. Predict the number of rows of `hot_rows`. Then write `toppings_per_table`. What are the SQL words for what you used?

Solution

Two rows: (Ada, american_hot_1, HotGreenPepperTopping) and (Ada, american_hot_1, JalapenoPepperTopping). A join returns every binding that makes the condition true; `exists` in (a) only asked whether there is one. In SQL: JOIN versus WHERE EXISTS.

```
toppings_per_table = set_of(o.table, count(topping)).grouped_by(o.table)      # (1, 7), (2, 8), (3, 3)
```

SQL: SELECT ..., COUNT(*) ... GROUP BY. The two joins are not written: `o.pizzas` and `ordered.toppings` are paths through the objects.

Task 3: Reasoning with Functions, Predicates and Rules

Notebook, part 3. Ben avoids milk. No object says "napoletana_1 contains milk"; it has to be derived: mozzarella is a cheese, cheese contains milk, the pizza has that topping.

(a) Functions. Complete `allergens`. What is `is_vegetarian(p)` when p is a variable instead of a pizza?

Solution

```
if any(isinstance(topping, CheeseTopping) for topping in pizza.toppings): found.add(Allergen.MILK)
if any(isinstance(topping, FishTopping) for topping in pizza.toppings):   found.add(Allergen.FISH)
```

Called on a pizza, a symbolic function returns its value. Called on a variable, it returns a piece of a query that is evaluated later, once for each binding. Last week's four functions are query vocabulary after one line each. This is procedural attachment: the meaning of the symbol is a program.

(b) Predicates and constraints. Complete IsSafeFor and write dangerous. What does it return? Why is this query a constraint and not a question?

Solution

```
def __call__(self) -> bool:
    return not (allergens(self.pizza) & self.guest.avoids)

dangerous = set_of(o, ordered).where(not_(IsSafeFor(ordered, o.guest)))
```

One row: the order of table 2 (Ben) with napoletana_1, which has mozzarella. Nobody is interested in the list as such; the kitchen needs it to be empty. A constraint is a query whose answer has to be empty, and a non-empty answer is useful: the binding says where the violation is. Databases call this an integrity constraint, SHACL calls it a validation report.

hasTopping is a stored relation (the list). IsSafeFor is a relation as well, a set of (pizza, guest) pairs, but it is computed. In a query the two look the same.

(c) Rules. Write the rules for fish and for gluten. How many warnings are derived? The predicate of (b) and the rules both find Ben's problem. When is each of them computed? Name one advantage of each.

Solution

```
fishy = a(Pizza)(toppings=a(FishTopping)()).from_(pizzas)
fish_warnings   = inference(AllergenWarning)(pizza=fishy, allergen=Allergen.FISH).tolist()
gluten_warnings = inference(AllergenWarning)(pizza=p, allergen=Allergen.GLUTEN).tolist()
```

13 warnings: 5 milk, 1 fish, 7 gluten. As a formula, the first rule is ∀x (Pizza(x) ∧ ∃y (hasTopping(x, y) ∧ CheeseTopping(y)) → ∃w (Warning(w) ∧ about(w, x) ∧ allergen(w, milk))). The body of a rule is a query; the head says what to build for each answer.

- **Predicate:** computed when it is asked (backward chaining, a view). Always up to date; paid for at every query.
- **Rule:** computed once and written down (forward chaining, materialisation; what HermiT did when it filled the inferred hierarchy). The conclusions are objects: they can be queried, joined, counted, stored. They become stale when the world changes (Task 4b).

(d) Rules with exceptions. Predict the label of each pizza. Which pizza gets none, and does that mean it is not vegetarian? After the ham on unclosed_1: which conclusion changed? What would an OWL reasoner have had to do?

Solution

margherita	napoletana	american_hot	mushroom	veggie	plain	unclosed
Vegetarian	**none**	Meaty	Vegetarian	Vegan	Vegan	Vegetarian

- napoletana_1 has fish: not vegetarian, so the default does not apply; no meat, so the alternative does not apply either. The rules are silent. Silence is not “no”: reading it as “not vegetarian” is one more closed-world step, which here happens to be right, and for “not meaty” as well.
- The refinement is an exception. Where it holds, the general conclusion Vegetarian is replaced by Vegan. Such default-and-exception trees are known as ripple-down rules.
- With ham, unclosed_1 changes from Vegetarian to Meaty. New information removed a conclusion: the reasoning is **non-monotonic**. Classical logic and OWL are monotonic: what is entailed stays entailed when axioms are added. The reasoner never has to retract “vegetarian”, because under the open-world assumption it never concluded it (Worksheet 2, Task 3c).

- The Label objects from before the ham still exist in Python. Removing conclusions whose reasons are gone is a task of its own, truth maintenance.

Task 4: Live Objects and Recorded Data

Notebook, part 4. So far the knowledge base was the *live* kitchen: Python objects that change while we look. At midnight the evening is *recorded* in a relational database (SQLite) and stays as it was.

(a) The same question in SQL. Why does the condition `topping.kind = 'CheeseTopping'` return nothing? What does the repaired query need that the EQL query did not?

Solution

The rows say MozzarellaTopping. That a mozzarella *is a* cheese is knowledge about classes (TBox); the tables hold facts about individuals (ABox). In Python the hierarchy is part of the program, so `a(CheeseTopping)()` fits a mozzarella. For SQL the hierarchy has to be stored as data (table subclass_of) and the query has to compute its transitive closure:

```
WITH RECURSIVE cheese(kind) AS (
   SELECT 'CheeseTopping'
   UNION
   SELECT subclass_of.child FROM subclass_of JOIN cheese ON subclass_of.parent = cheese.kind)
SELECT DISTINCT pizza.name FROM pizza JOIN topping ON topping.pizza_id = pizza.id
WHERE topping.kind IN (SELECT kind FROM cheese)
```

Recursion is what Datalog adds to relational algebra. SPARQL has it for this case as the property path `rdfs:subClassOf*`.

(b) The world moves on. The cook grates parmesan over Ben's veggie_1 by mistake. Predict what each source says about cheese on veggie_1. Which one would have protected Ben?

Solution

1. query `cheesy` of Task 2, live objects	2. the record	3. milk warnings of Task 3(c)
veggie_1 is cheesy	no parmesan anywhere	no warning for veggie_1

- Only the live query protects Ben. It was written before the mistake and is evaluated against the state at the moment of evaluation. Nothing was reloaded.
- The record is a snapshot, and that is its purpose: an inspector asks what was served on Thursday, not what is on the counter now. It answers “what was true then”.
- The warnings are derived facts, a small record of their own. After the change they are wrong until the rule runs again. *When* to re-derive is the hard part of forward chaining, and a robot, whose world changes continuously, faces it all the time.

(c) What is missing? Try to write `dangerous` in SQL over the record. What exactly is missing?

Solution

Not expressive power, but data. A SQL query can only use what is a row in a table. The record has no table for what a guest avoids, and “cheese contains milk” was inside the Python function `allergens`. With two more tables (avoids, contains) the query is possible: five joins and a recursive expression (see the solution notebook; result: table 2, Ben, napoletana_1, milk). In EQL it was one line because the query follows object references (`o.guest.avoids`) and calls the function.

The price: the database holds millions of rows on disk, uses indexes, survives a power cut and serves many users at once. A query over Python objects does none of this. KRROOD's ORMatic connects the two: it generates the tables from the dataclasses and translates a part of EQL to SQL.

(d) Four query languages. Fill in the columns SQL and EQL from what you saw today. SPARQL (which you would use on pizza.owl) and Datalog/Prolog follow in the discussion.

Solution

	SQL	SPARQL	Datalog / Prolog	EQL
What is queried?	tables of rows in a database: the record	a graph of triples, e.g. pizza.owl with individuals	facts and rules of a program	the objects of a running Python program
From a pizza to its toppings	join on a foreign key	triple pattern `?p :hasTopping ?t`	shared variable: `has_topping(P, T)`	attribute path `p.toppings`, nested description
"A mozzarella is a cheese" comes from	nowhere, unless stored as rows and closed with WITH RECURSIVE	the graph: path `rdfs:subClassOf*`, or a reasoner behind the query	a rule: `cheese(X) :- mozzarella(X).`	the class hierarchy of the program
"Not" means	no such row (NOT EXISTS): closed world	no such triple in the graph (FILTER NOT EXISTS), although OWL itself is open-world	not provable: negation as failure	no such object in the domain: closed world
Rules	views; recursion only with WITH RECURSIVE	none in the language (CONSTRUCT builds triples; rules come from OWL, SWRL)	the heart of the language, recursive	`inference`, rule trees with exceptions
Your own function in a query	only if registered inside the database	built-in filter functions, vendor extensions	every predicate is a program	any Python function or predicate

None of the four is "the best". They sit at different points between two wishes: to ask rich questions about a world that is changing now, and to keep large amounts of what was once true safely for later. A robot needs both, and its memory of past episodes is a record that it queries with the same descriptions as its present.

(e) Your turn. You are building a robot that tidies a kitchen. In words (or in EQL, with classes you invent), give one query of each kind and say what its domain is: a **question** the robot asks its knowledge base, a **description** of something that does not exist yet and that the robot has to produce, and a **constraint** that must never have an answer.

Solution

Many answers are good; three examples.

- **Question:** "Which cups stand on the table and are not clean?" Domain: the objects the robot currently believes to exist, a live knowledge base that perception keeps changing. `a(Cup) (supported_by=the(Table)(), clean=False)`
- **Description to be produced:** "a pose in the dishwasher rack where the cup stands upright, is reachable and touches nothing". No such pose is stored anywhere. The domain is a continuous space, the description is underspecified, and the robot has to generate a pose that fits (the bonus cell of the

notebook does this for a pizza).

- **Constraint:** “a knife in the gripper while a person is closer than one metre”. Domain: the state of the world at every moment. The answer has to stay empty, so the query is not asked once but watched; it is a monitor.